

Lab-4

Task 1a:

This task is about creating adjacency matrix. So at first we will create a matrix of 0 which row and ~~and~~ column are vertices + 1. Then we will just add the weighted values in the matrix using a loop. Since it's a directed graph so $[u][v] = w$. Otherwise we had to $[v][u] = w$ as well.

Task 1b:

This task is about creating adjacency list. At first I have created a nested list in which number at list are vertices + 1. ~~I created~~ After that I took another list to separate v and w . Then using a loop I inserted the v and weight in the certain vertices as a tuple.

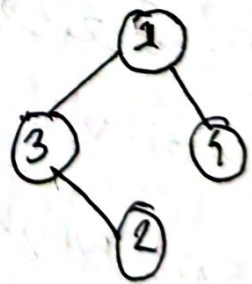
Graph Creation

Creation of graph will be needed in each task. So, I will create a graph class which will basically create an adjacency list.

Here I have used ~~graph~~ as a dictionary as a global variable as it is easier to iterate. The function add edge will create separate list for u and v if it's not in the dictionary. If it is in the then it will ~~add~~ insert the (u,v) and (v,u).

Task-2 BFS Traversal

I will use visited to check if it's already visited or not. ~~we~~ Use deque to pop the left most. As we pop the value and it's not visited add the value in our path list. We have to check if the adjacent



visited 1, 3, 4, 2

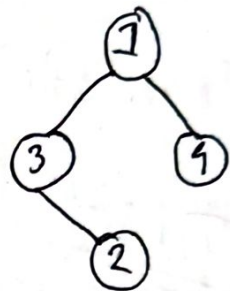
queue 1, 3, 4, 2

path 1, 3, 4, 2

list value is value is visited or not. If it's visited we continue. If it's not we add it to our queue.

Task-3: DFS traversal

DFS traversal is easier: We will create a visited set and a path, and add our source value. Now check if its adjacencies are ~~not~~ visited. If it's visited then continue, if it's not the dfs again ~~not~~ where means adding them to visited and path and checking their adjacent.



visited: 1, 3, 4, 2

path: 1, 3, 4, 2

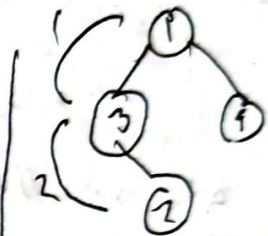
Task-4

Here we used visited set and a stack to check the repetition of value. 'visited' is set to keep the track of visited nodes and 'stack' is a set to keep the track of nodes in the path it is being explored. If the value is already visited it will return false meaning it's ^{not a} cycle. If it's ~~not~~ in the

stack will return true. The outer loop iterates over all nodes in the graph and checks for cycle from each node. If a cycle is found the method returns True otherwise False.

Task-5

Here we have to take three parameters, graph, start and end. ~~Explain~~ We will take the visited set to check if it's visited and queue which will take tuple. As it starts with the start city and its path is also the start. The loop will run until the queue is empty. In each iteration it will dequeue a tuple (current city, path). If it is our desired city it will return the path. Or else it adds the current city to the visited. and checks its neighbours. For each unvisited neighbour it appends a tuple (neighbour, path + [neighbour]) to the queue.



Task 6

First we read the node and edge number
Now we make a adjacent matrix using
grid variable. It will help in iteration.

It will look like

[['.', '.', 'D']

['D', '.', '.']

[['.', 'D', '.']

['#', '#', '.']]

input

4 3

..D

D..

.D.

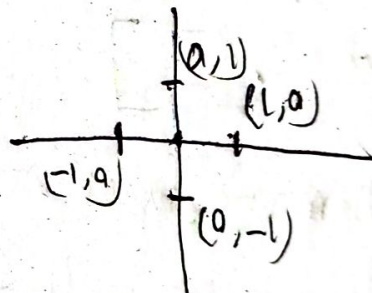
##.

3

The max diamond function will return the maximum number of diamonds that can be collected. Here we will take another matrix of zeros which will be visited matrix that will check if that certain index is visited or not. Will use a nested loop for iteration where it will check if it's '.' and not visited. then pass it to the dfs function.

The dfs function takes four parameters. They are grid, current row index, current column index and matrix and the

visited cells. At first it checks if there is an obstacle or it's already been visited. If it is true then returns 0, which means no diamond from this cell. If $grid[r][c] = '0'$ then we make the visited 1 and diamond number 1. If it's not diamond in zero and $visited[r][c] = 1$ as it's ~~already~~ now visited. The outer loop iterates from direction right, down, left up.



For each direction it calculates the new row and column and recursively calls the DFS function. At last it will return diamonds, which will be checked in max diamond function that if diamond count is bigger than the diamonds collected. Then return our maximum output.